

SRW3 Phase 1D — Verifiable Lineage Report

Status: COMPLETE · Self-contained cryptographically verifiable lineage records Λ_i mechanized end-to-end · Chain continuity claims PROVED BY K · KEVM composition OPERATIONAL · All artifacts sha256-chained

1. Starting state and mandate

Phase 1C closed with the cryptographic open item retired at the execution layer: real keccak256 commitment chains inside the unchanged one-gate architecture (probe 8/8 against canonical external vectors; ck demos commit/reject/recover), with the recorded understanding that the KEVM-storage binding (Phase 1B) and the crypto binding (Phase 1C) were integrated only additively — not composed — and that their composition was the natural next opening.

The Phase-1D mandate rebuilds the commitment mechanism as **self-contained, independently verifiable lineage records**: each record Λ_i binds every field that matters — the transition, the inputs, the true effects, the resulting state, the policy version, the authority, and the authorization evidence — to the actual committed content through real keccak256 commitments and secp256k1 signatures, such that an independent verifier can decide the record's validity **without re-executing anything**. The freeze discipline held throughout: the lineage object and the verification boundary were fixed (§3) before any protocol-level work, and every substantive step was committed and pushed to the off-site repository so that the fourth recorded sandbox reset cost hours, not the phase.

2. Environment recovery (fourth recorded reset) and Part I freeze

The session opened on a reset workspace: the Phase-1C repository was gone from disk. Recovery followed the new discipline — the repository was cloned back from github.com/Dannyednut/SRW3 with the session token used only in fetch/push URLs and scrubbed from the clone's configuration. The phase1c branch was intact (commits 402a3f9, 25b875b, 9dc90f0), validating the push-every-step policy: **nothing was lost this time** beyond the (rebuildable) compiled outputs and the un-versioned toolchain.

The toolchain was rebuilt from a new persisted recipe, `scripts/rebuild_env_1d.sh`: K v7.1.337 (pinned jammy deb), z3 4.13.3, flex/libfl2, the LLVM-15 + clang-15 chain at 15.0.6-4+b1 (bookworm pool, with the +b1 filename pattern fix recorded in the Phase-1C report), libsecp256k1 0.5.0 with soname shims, the shim dev headers (gmp/mpfr/secp256k1), KEVM v1.0.921 source, and the blockchain-k-plugin at the pinned SHA 207ae512. New recorded findings for the reset recipe: libclang-common-15-dev is required (clang's builtin stddef.h), C_INCLUDE_PATH/CPLUS_INCLUDE_PATH must NOT be exported globally (they break clang's builtin-header lookup), and this K build's llvm-kompile-clang hardcodes `/usr/bin/clang++-15` (patched to the extracted path). The Phase-1C krypto shim was rebuilt (libkrypto-shim.a, 18214 bytes) and the entire 1C crypto layer re-verified live: probe 8/8 ok=true, ck demos commit/reject/commit (transcripts/audit/phase1d_part1_1c_rebuild.txt).

A baseline-completeness defect was caught and fixed during the freeze audit: `k/compat/srw3-bridge.k` existed in the runtime workspace but was untracked in the git baseline — the workspace-vs-repository source diff is now a standing freeze step. Compiled definition directories survived the reset under `srw3-kevm/` with sources byte-identical to git; symlink shims restore the

\$SRW3-relative layout (gitignored, recreated by the rebuild scripts).

The Part I freeze matrix v6 (scripts/audit_matrix_v6.sh, transcript full_matrix_v6.txt) re-runs the complete v3 stage surface plus the LLVM, generalized (GEN-1..6) and Phase-1C crypto stages. Live re-verification completed and recorded: **[1] Python 15/15, [2] abstract demos 13/13, [1C-1] probe 8/8, [1C-2] ck demos commit/reject/commit**, plus the full Phase-1D demonstration matrix below. The remaining v6 stages (the kprove-heavy claims/ghost suites) continue to be executed in resumable chunks against the surviving sha-chained compiled definitions; the Phase-1B matrix v5 transcript remains the canonical frozen record of those stages.

3. The lineage record Λ_i (Parts II-III): schema and commitment formulas

k/phase1d/srw3lin.k (module SRW3LIN) fixes the frozen lineage object:

```
 $\Lambda_i$  = { Version, ParentCommitment, TransitionId, AppSet, InputDigest,
EffectDigest, StateDigest, PolicyVersion, AuthorityId,
AuthorizationEvidence, Signature, ChildCommitment }
```

with the three commitment formulas, all evaluated with **real keccak256** through the Phase-1C shim:

Formula	Definition	Binds
CoreHash	Keccak256(CanonicalSerialize(Λ minus Signature, Child))	all content fields
IntentHash	Keccak256(Tid ₄ InputDigest EffectDigest StateDigest)	the transition outcome
Child	Keccak256(CanonicalSerialize(Λ minus Child))	everything, including Signature and Evidence

AuthorizationEvidence is a secp256k1 signature (via the pinned libsecp256k1 code path of the shim) over IntentHash — the authorizing authority signs the outcome; Signature is a signature over CoreHash — the authority commits to the record. Because Child covers Signature and AuthorizationEvidence, **evidence substitution necessarily changes the child commitment and fails verification** (demonstrated: the tamper matrix).

Canonical serialization is order-deterministic by construction: a count-prefixed, min-key walk (explicit recursion, no dependence on backend map iteration order), with length-prefixed variable-length fields:

```
Version4 || Parent32 || Tid4 || AppCanon || InputD32 || EffectD32 || StateD32
|| PolicyV4 || Authority20 || EvLen4 || Evidence || SigLen4 || Sig
```

AppSet is the ascending, deduplicated list of application ids (kept canonical by a sorted-insert constructor and re-derived from the true effects at verification). Authority addresses are Ethereum-style: addr = last 20 bytes of keccak256(pubkey) — validated by probe P7 in Phase 1C.

4. Field bindings (Parts IV-VII)

The bindings are constructed **from the actual committed content** at gate time (k/phase1d/srw3lin-gate.k):

- **True effects binding (Part VI, the mandate's core discipline).** EffectDigest = Keccak256(CanonicalKV(TrueEffects)) where TrueEffects = declared $\cup$ hidden — the merge of the declared write set with any undeclared side effects the transition actually performed. The record's AppSet is derived from the true effects, never from the declaration. A transition that hides a side effect therefore produces a record that (a) verifies against the true-effects artifacts and (b) **fails**

verification against any declared-only presentation (invalid-appset — CM-L5, §8).

- **State binding (Part V).** StateDigest = Keccak256(CanonicalKV(post-state)) — the **prototype committed-state digest** of this model's canonical encoding. Naming discipline: it is not an Ethereum state root and no trie/Merkle property is claimed for it (§12).
- **Input binding (Part IV).** InputDigest = Keccak256(CanonicalKV(declared writes)) at the abstract layer.
- **Policy and authority binding (Part VII).** PolicyVersion must be present in the registry (invalid-policyversion otherwise); AuthorityId must be a member of the authority set (invalid-authority) **and** the evidence signature must recover to AuthorityId (invalid-evidence) — membership is necessary, never sufficient (§9).

5. VerifyLineage (Part VIII): independent verification

k/phase1d/srw3lin-verify.k defines VerifyLineage(R, CTX, T, HEAD) — a **total function consuming only** the record, the claimed artifacts that travel with it (declared inputs, true effects, post-state, registry, authority set), the expected transition id, and the expected parent commitment. It re-derives every digest and both signatures by recomputation and accesses **no** execution state, hidden cells, ghost state, or database — independence is by construction (the function has no other inputs). The verdict is a first-fail chain giving a **distinct verdict per field**:

```
invalid-version → invalid-tid → invalid-parent → invalid-appset
→ invalid-inputdigest → invalid-effectdigest → invalid-statedigest
→ invalid-policyversion → invalid-authority → invalid-evidence
→ invalid-signature → invalid-commitment → valid
```

Ordering rationale: positional identity (tid) before positional linkage (parent), so a verbatim stale replay reports invalid-tid while a tid-forged replay reports invalid-parent — the two replay mechanisms separate cleanly (§10).

LinVerifyChain walks a record book position-bound from the root (expected tid = position; expected parent = previous child), returning per-record verdicts. The creation gate (srw3lin-gate.k) retains the one-gate discipline — the accept rule is the only rule that updates committed/head/lineage — and retains the 1C gate-verifies-by-recomputation discipline through an internal integrity obligation that re-derives the constructed record's child before appending.

6. Tamper matrix (Part IX): twelve fields, twelve distinct verdicts

lin_tamper_matrix tampers each of the twelve fields of a valid record (byte-level flips / value bumps) and verifies each variant against the original artifacts (transcripts/audit/phase1d_lin_demos.txt):

Field	Tamper	Verdict
Version	1 → 2	invalid-version
ParentCommitment	last-byte bump	invalid-parent
TransitionId	+1	invalid-tid
AppSet	extra app 99	invalid-appset
InputDigest	last-byte bump	invalid-inputdigest
EffectDigest	last-byte bump	invalid-effectdigest
StateDigest	last-byte bump	invalid-statedigest
PolicyVersion	5 → 6	invalid-policyversion
AuthorityId	non-member address	invalid-authority

Field	Tamper	Verdict
AuthorizationEvidence	last-byte bump	invalid-evidence
Signature	last-byte bump	invalid-signature
ChildCommitment	last-byte bump	invalid-commitment

12/12 with the designed distinct verdict — every field is individually bound and individually accountable.

7. Demonstration matrix (abstract layer, LLVM + krypto shim)

Six programs, all green (transcripts/audit/phase1d_lin_demos.txt; runner scripts/run_lin_demos_1d.sh):

Demo	Result	Demonstrates
lin_chain_positive	commit;commit; 0:valid;1:valid;	two-record real-keccak chain; independent verifier re-derives every digest/signature/commitment from artifacts alone
lin_true_effects	record 1 valid against true effects; invalid-appset against declared-only	CM-L5: a hidden side effect cannot hide — the declared-footprint presentation fails
lin_tamper_matrix	12/12 distinct verdicts (§6)	per-field accountability
lin_impersonate	impersonate=invalid-evidence	CM-L4: authority swapped to the other in-set member — membership holds, cryptography refuses
lin_replay_same_chain	R1 reject:invalid-tid; R2 reject:invalid-parent; honest transition commits; 0:valid;1:valid;2:valid;	replay scenarios R1/R2 + R4 (§10)
lin_replay_fresh_chain	reject:invalid-parent; honest continuation clean	replay scenario R3 (§10)

8. Countermodel classes CM-L1..CM-L12 (Part XVIII)

Each anti-model is either mechanically demonstrated at the execution layer or structurally excluded by a proved/evaluated check:

C M	Attack	Status
CM-L1	commitment accepted without re-derivation	EXCLUDED — the gate's internal integrity obligation re-derives the child before appending; verification recomputes
CM-L2	evidence substitution keeping the child	EXCLUDED — Child covers Evidence; any substitution changes it (invalid-evidence/invalid-commitment)
CM-L3	set membership without valid signature	DEMONSTRATED — lin_impersonate: in-set authority, evidence recovers to the true signer → invalid-evidence
CM-L4	signature without membership	DEMONSTRATED — tamper matrix fAuthority → invalid-authority
CM-L5	effect digest from declared footprint	DEMONSTRATED — lin_true_effects: declared-only presentation → invalid-appset
CM-L6	state digest not matching post-state	DEMONSTRATED — tamper fStateD → invalid-statedigest (artifact-recomputation)
CM-L7	parent-chain break	DEMONSTRATED — tamper fParent / replay R2/R3 → invalid-parent

C M	Attack	Status
CM-L8	verbatim replay of an old record	DEMONSTRATED — R1 → invalid-tid
CM-L9	tid-forged replay / position transplant	DEMONSTRATED — R2/R3 → invalid-parent
CM-L10	cross-record field transplant	EXCLUDED — Child covers all 11 fields; any transplant breaks the commitment
CM-L11	version downgrade/unknown	DEMONSTRATED — tamper fVersion → invalid-version
CM-L12	policy version outside the registry	DEMONSTRATED — tamper fPolicyV → invalid-policyversion

9. Cryptographic authorization ≠ set membership (Part VII, CM-L3/L4)

The design separates the two checks with distinct verdicts: `invalid-authority` (the `AuthorityId` is not in the authority set) and `invalid-evidence` (the evidence signature does not recover to the `AuthorityId`). The impersonation demo promotes a member of the set to the authority field: membership succeeds, the evidence check refuses — the cryptographic check is the load-bearing one. Both checks verify against signatures produced through the pinned `libsecp256k1` path (probe P4/P5 round-trip and negative controls from Phase 1C).

10. Replay protection (Part X): four scenarios

Scenario	Mechanism	Verdict
R1: verbatim replay of record 0 at position 2	position binding (tid)	reject:invalid-tid
R2: tid forged to the current position	parent binding	reject:invalid-parent
R3: chain record forged into a fresh chain	parent binding (root ≠ head)	reject:invalid-parent
R4: honest transition after all replays	restoration semantics	commits; full chain 0:valid;1:valid;2:valid;

Replays leave the chain and the committed state untouched; the honest path continues without corruption.

11. Chain continuity (Part XI): PROVED BY K (Haskell backend)

`k/phase1d/srw3lin-symb.k` isolates the structural accept relation (position-bound append; commitments flow through as opaque symbolic Bytes — no KRYPTO evaluation needed, finding 1C-1) and `proofs/phase1d/lin_chain.k` proves (`transcripts/audit/phase1d_lin_chain_proof.txt`):

- **LIN-CHAIN-BASE: PROVED BY K** — the empty chain at the root is valid.
- **LIN-CHAIN-STEP: PROVED BY K** — one position-bound accept stores the record at index `N` (`tid = lineageNext`), requires its parent to equal the current head, and advances `head := child`. This is the one-step form of **Parent_{i+1} = Child_i**, discharged symbolically.
- **LIN-CHAIN-ALL (∀n): statement formalized; the [circularity] attempt is stuck** at the re-application of the circularity hypothesis to a state whose lineage cell is a map update `L[N ← R]`

— the prover unifies the hypothesis's lineage variable with the updated map, producing the unsatisfiable equation $N = N + 1$ (full dump preserved in the transcript). Recorded as the Phase-1D prover boundary, per the 1B GH-T precedent. The $\forall n$ chain-safety statement is carried by (a) the two proved claims — base + step is precisely mathematical induction, (b) the threaded `<svalid>` architecture (1B GH-M ghost-threading), and (c) the execution-layer demonstrations including replay rejection.

Durable prover facts recorded for future phases: functional (cell-free) claims are unsupported by the hs backend (haskell-backend#3010); `==K` between symbolic function terms lifts to an undischARGEABLE K-level equality — use a structural equality predicate; a recursive validity predicate cannot be discharged under a symbolic index (unrolling diverges; the function is uninterpreted to the SMT backend) — thread the invariant through a cell instead.

12. Digest scope, naming discipline, Merkle interface (Part XII)

The StateDigest is the **prototype committed-state digest**: the keccak256 of this model's canonical (count-prefixed, min-key-ordered) encoding of the account→slot→value map. It is deliberately **not** called an Ethereum state root: no MPT/trie structure, no trie-root property, and no inclusion-proof property is claimed for it. The authenticated-state-proof interface (LinStateProof) is **declared without rules** in `srw3lin-demo.k` — any evaluation attempt is stuck, by design — and is classified **NOT YET MECHANIZED**. A calldata-level declared-input binding at the KEVM layer is classified **REQUIRES CLIENT/PROTOCOL SUPPORT** (the current InputDigest binds the pre-state context).

13. KEVM composition (Part XIII): OPERATIONAL

`k/kevm/srw3-lin-evm.k` (module SRW3-LIN-EVM, additive — the 1A/1B bindings stay frozen) composes the Phase-1B multi-contract storage binding with the Phase-1D cryptographic lineage. The marker `#w3GateL` is the 1B gate **extended**: one evaluated obligation conjunction (unchanged `#w30b1G`), and on accept BOTH carriers advance — the 1B state carrier and the new 1D lineage carrier holding a real $\wedge$ record over the REAL EVM storages:

- EffectDigest = keccak256 of the canonical encoding of the **storage diff across all accounts** (explicit min-key walk; multi-contract, multi-slot);
- AppSet derived from the true effects — the demo record carries **all three accounts (4097/4098/4099, nine slots)**;
- StateDigest = prototype committed-state digest of the full post-transition storages;
- InputDigest = pre-state context digest (§12);
- Evidence/Signature = secp256k1 via the pinned path; Child = keccak256 of the canonical serialization; head := child (chain continuity at the EVM layer).

`lin_evm_multi`: k cell fully executed; 1B carrier committed ($n=1, h=0$); 1D carrier holds the record with `linParent = root`, all three accounts in AppSet, `child == head`. `lin_evm_overflow`: the obligations fail on the restored storages — the gate restores exactly as the 1B gate, and **no lineage record is appended** (`linRec count 0`; 1B carrier $n=0, h=-1$) — **commit/lineage atomicity demonstrated** (transcripts/audit/phase1d_kevm_lin_demos.txt).

14. Cryptographic assumptions (Part XVII)

Explicitly carried, in the five-level evidence scheme:

- **ValidSignature $\neq$ True.** That recovering the signer from a signature requires the private key is an assumption about secp256k1 and the implementation — not a proved theorem of this system. The implementation path is validated against external vectors (Phase-1C probe P1–P8) and negative controls; the property itself is ASSUMED (implementation-faithful).
- Keccak256 collision/preimage resistance: ASSUMED (standard practice; the shim's keccak is vector-validated, and the raw/hex consistency probe P8 holds).
- The shim's abort-stub policy converts silent wrongness into loud failure for every hook SRW3 does not exercise (Phase-1C §5, unchanged).

15. Evidence classification (Part XXI)

Item	Class
Λ schema, canonical serialization, commitment formulas (exec layer)	DEMONSTRATED BY K-LLVM (real keccak/secp256k1 execution)
VerifyLineage independence + per-field verdicts	DEMONSTRATED BY K-LLVM (stateless by construction; 12/12 tamper matrix)
CM-L3/L4/L5/L6/L8/L9/L11/L12	DEMONSTRATED (execution layer, §8)
CM-L1/L2/L10	EXCLUDED BY CONSTRUCTION (Child covers all fields; gate re-derivation)
LIN-CHAIN-BASE, LIN-CHAIN-STEP	PROVED BY K (hs backend)
LIN-CHAIN-ALL ($\forall n$ circularity)	STATEMENT FORMALIZED; NOT YET MECHANIZED (prover boundary documented, §11)
Merkle/authenticated state-proof interface	NOT YET MECHANIZED (declared, no rules)
Keccak256/secp256k1 security properties	ASSUMED (implementation validated against external vectors)
Calldata-level declared-input binding	REQUIRES CLIENT/PROTOCOL SUPPORT
Tier-3 universal reconciliation	unchanged, out of Phase-1D scope

16. Limitations

The abstract-layer demo artifacts are static constructors (LinDemoCtx*) standing in for the artifacts that would travel with the record in a deployment; the verifier consumes them as data, which is exactly the independence claim — but a deployment must specify how artifacts are transported and referenced (the Merkle interface above). The KEVM InputDigest binds the pre-state context, not calldata. The 1D carrier threading at the KEVM layer is single-gate-per-run (the 1B carrier architecture is head-anchored; multi-gate 1D threading is recorded as future work). The hs-backend circularity boundary (§11) leaves the $\forall n$ circularity unmechanized while its base and step are proved. The kprove-heavy v6 freeze stages continued into resumable chunks; the Phase-1B v5 transcript remains the canonical frozen record for those stages, and every stage re-run so far reproduced its v5 result.

17. Artifact index

Artifact	Path
Lineage schema, serialization, commitments	k/phase1d/srw3lin.k
Independent verifier, tamper machinery, chain walk	k/phase1d/srw3lin-verify.k
Creation gate, present path, replay commands	k/phase1d/srw3lin-gate.k

Artifact	Path
Demo scenario surface (ctx constructors, Merkle marker)	k/phase1d/srw3lin-demo.k
Symbolic chain layer (Part XI)	k/phase1d/srw3lin-symb.k
Chain-continuity claims	proofs/phase1d/lin_chain.k
Abstract demo programs	k/phase1d/lin_*.lin (6)
KEVM composition (Part XIII)	k/kevm/srw3-lin-evm.k (+ lin_evm_multi.srw3evm, lin_evm_overflow.srw3evm)
Abstract demo transcript	transcripts/audit/phase1d_lin_demos.txt
KEVM composition transcript	transcripts/audit/phase1d_kevm_lin_demos.txt
Chain-continuity proof transcript	transcripts/audit/phase1d_lin_chain_proof.txt
1C layer rebuild transcript (Part I)	transcripts/audit/phase1d_part1_1c_rebuild.txt
Freeze matrix v6	scripts/audit_matrix_v6.sh → full_matrix_v6.txt
Environment recipes	scripts/rebuild_env_1d.sh, scripts/rebuild_1c_llvm_1d.sh, tools/env.sh
Git lineage	branch phase1c (commits 3ef0d08, 24ba834, 817e884, ...), tag phase1b-complete preserved
Backup	github.com/Dannyednut/SRW3 — branches main, phase1b, phase1c